Admin SPA on the order endpoints

Report of the work done on branch feat/gateway-api, 10 October 2026

Status	Done and verified; committed with this report.
Scope	The auctions, orders and payouts screens of the admin SPA (`gateway-admin`) now read the order module's admin endpoints through the admin edge instead of local mock data. Types follow the backend rows; nothing is reshaped on the way.
Tests	47 SPA tests pass (28 before), type check and translation parity (457 keys in en, da, sv) green; the admin jar with the SPA built and run against the order service; checklist in section 5.

1. Starting point

The three trade screens existed behind a feature flag and ran on mock data served by a service worker. Their types were a sketch of the backend written before the endpoints existed: amounts in cents, field names such as `announceTitle` and `increment`, a bid field `placedAt`, an auction status `SNAGGED`, and an order detail bundling payment, shipment and a timeline. The endpoints added in the previous round answer the read-model rows as they are (decimal amounts, `title`, `bidIncrement`, `bidTime`, `CLOSED_SNAGGED`), payment and shipment behind their own paths, and a payouts answer of totals plus a page. Two more problems were found on the way:

- **The dispute resolution sent the wrong field.** The screen sent `{resolution, reason}`; the backend takes `{resolution, note}`. Both staff mutations answer the domain record in cents, not the row, so the screens must refetch rather than use the answer.
- **The mock switch would have hijacked the real calls.** The worker started whenever a phase-2 flag was on outside production. With the trade flag on, every real trade call in development would have been intercepted.

2. Decisions

Decision	Consequence
Types follow the backend rows	The clients declare `AuctionRow`, `BidRow`, `OrderRow`, `PaymentRow`, `ShipmentRow`, `PayoutRow` and `PayoutTotals` field for field; money goes through the decimal formatter; the status lists match the backend, including `CLOSED_SNAGGED` and `DECLINED`.
Order detail is three calls plus a derived timeline	The row, its payment and its shipment each from its own endpoint; a 404 on the last two means "none yet". The timeline is built in the screen from the rows' timestamps; no backend table, no event-store read.
Mutations send what the backend takes and refetch	Resolve body `{resolution, note}`; answers ignored; lists and rows refetch through the query tags.
Mocks only on request; trade on by default	The worker starts only with `features.mocks`; fixtures rewritten to the real shapes so mocked mode shows the same screens; the mock banner shows only then. `features.trade` defaults on, `features.chat` stays off.
Small filter additions	Title words and seller id on auctions and orders, buyer id on orders, seller id on payouts: exactly what the endpoints accept.

3. What changed

Admin SPA (`gateway-admin/src/main/webapp`)

Area	Files	Change
Clients	`app/shared/api/{auctionsApi,ordersApi,payoutsApi}.ts`, `orderTimeline.ts`	Row types, payment and shipment queries, the resolve body, the timeline derivation.
Screens	`app/modules/trade/*`	Four pages on the new fields and the decimal formatter; `TradeFilterBar` and `tradeFilters.ts`; the bids panel shows increment, reserve, buy-now, auto-extend, close time, max amounts and the cancelling staff member, and cancels only while OPEN; payouts show five totals with counts, fees collected and pending, the order status under the payout status and the seller gate.
Flags and mocks	`app/config/features.ts`, `app/mocks/{handlers,tradeFixtures,browser}.ts`, `app/index.tsx`	Mocks only with the flag; trade default on; fixtures in the real shapes with the real filters and 404 rule; payout lines derived like the backend does.
Shared	`usePaging.ts`, `StatusBadge.tsx`, `baseQuery.ts`	One URL update for several filter keys; badge colours for the new statuses; absolute same-origin base URL.
Translations	`app/i18n/{en,da,sv}/{trade,common}.json`	Status names, filter labels, timeline steps, payout labels, the note field; parity check green.
Tests	`features.test.ts`, `orderTimeline.test.ts`, `tradeFilters.test.ts`, four page tests, `testUtils.tsx`	Page tests render through the real store and base query against msw/node serving the browser fixtures. Jest got the export-condition option, fetch polyfills from `undici` (new dev dependency).

Documentation

- `doc/frontend-edges.md` §6 and §12 updated, new §13 (what was wired, how the flags work, the tests) and §13a (checklist).
- Memory of the plan updated to done.

4. Verification

Check	Result
`npm run typecheck`	clean
`npm run i18n:check`	OK: 11 files, 457 keys in en, 424 literal keys in code, da and sv in parity
`npm test`	13 suites, 47 tests pass (28 before this work)
`mvn -pl db-reactive,gateway-admin -P webapp -DskipTests package`	jar contains static/index.html, assets/js/main.*.js and mockServiceWorker.js

5. Checklist against the running services

Setup: live MySQL, `docker/cqrs` (EventSourcingDB, Kafka 9094), the order service from its jar on 8083 (service token enforced, so every call went through the edge), `gateway-admin` from the jar built above with `--spring.profiles.active=dev` on 8081. The development database held one OPEN auction with one bid and one PROPOSED fixed-price order from the 27 September stack run, nothing else. Endpoint calls with curl and a bearer token from `POST /auth/login`; screens driven in headless Chromium.

Step	Result
`GET /api/admin/auctions`	200, one row: title, status OPEN, decimal `currentPrice 120.0`, `bidIncrement 10.0`, `autoExtendSec 120`; `status=CANCELLED` gives total 0; `size=500` is capped and answers 200

Step	Result
`GET /api/admin/auctions/{announceId}`	200, `{auction, bids}` with one ACTIVE bid carrying `bidTime` and `amount 120.0`
`GET /api/admin/orders`	200, one row: PROPOSED fixed-price order at 1250.0 with buyer 14 and seller 10; `status=DISPUTED` and `q=zzzz` give total 0
`GET /api/admin/orders/{id}`, `/payment`, `/shipment`	row 200; payment 404 and shipment 404 (none yet for a PROPOSED order)
`GET /api/admin/payouts`	200, every total 0 with 0 orders, currency DKK, empty page (a PROPOSED order holds no money); `status=FAILED` is 400
Edge	without a token 401; `GET /` and `GET /payouts` answer index.html; `main.*.js` 200 with the trade flag compiled on and the resolve body `{resolution, note}`
Auctions screen	one row, no problem alert, no mock banner; clicking it opens the bids panel with the increment line and the bid
Orders screen and detail	one row linking to its detail; the detail shows the summary, "No payment started", "No shipment requested" and an empty timeline (a PROPOSED order has no timestamp yet); no alert
Payouts screen	five tiles at DKK 0.00 with 0 orders, the fees line, the empty state; no alert
Filters	status DISPUTED applied on the orders screen writes `?status=DISPUTED&page;=1` and shows the empty state

6. Left for later

- The support queue and bulletins still run on the fixtures behind `features.chat`; wiring them to messaging's staff endpoints is its own plan.
- Trade data only appears after a flow through the customer edge; the development database held one PROPOSED order and one OPEN auction, so the rich states (disputes, payouts) were exercised in the page tests over the fixtures, not against the backend.
- Roles are not checked inside the backends; the edge enforces `ROLE_ADMIN` on `/api/**`.